

COMP6030 Group Project Technical Report

London Underground “What-If” Simulation

Members:

Sinead Parkinson – sop3@kent.ac.uk

Sattyaj Paul – sp972@kent.ac.uk

Saf Sikder – ss2766@kent.ac.uk

Jaden Toon – jt681@kent.ac.uk



School of Computing

University of Kent

Abstract

This report presents a What-If simulator for the London Underground. Most journey planners only route across the network as it is right now. Our tool goes further: a user can close a station, suspend a line or block a single connection and immediately see which journeys change and by how much.

We model the network as a weighted graph in Neo4j. The topology comes from the TfL Unified API, and the edge weights come from TfL timetable data. Routing uses Dijkstra's algorithm, which we made aware of line changes and run in the browser, so a route re-plans the moment a closure is toggled. The interface is built with React, Next.js and Leaflet. It draws the network on a real map and overlays live line status, station disruptions and train positions, switching to simulated train movement when the live feed is unreliable.

Routing completed in single-digit milliseconds across the full network, inside the ~100 ms threshold at which an interface feels instantaneous to the user. A user study with 21 participants led to five usability changes during the final sprint, with a sixth left to future work.

1. Introduction

The London Underground is one of the most heavily used urban transport networks in the world, supporting millions of journeys across a dense and interconnected system of lines and stations [1]. Disruptions such as delays, station closures, suspended services and blocked connections can therefore have a significant impact on passenger journeys. Although existing applications such as TfL Go [2] and Google Maps [3] provide live status information and route recommendations, they are primarily designed for immediate journey planning and offer limited support for exploring hypothetical disruption scenarios.

This project develops an interactive What-If simulator for the London Underground network. The system allows users to create disruption scenarios by closing stations or suspending lines, then observe how these changes affect route availability and journey times. Rather than simply presenting the current network status, the application is an exploratory tool. It is aimed primarily at commuters exploring how disruption affects their own journeys, with a secondary, potential application for planners and researchers studying network resilience and the wider knock-on effects of disruption. Fig. 1 shows the simulator's main interface.



Fig. 1. The What-If simulator showing the London Underground network.

The project applies core computer science concepts to a real world transport domain. The Underground is modelled as a graph, with stations as nodes and connections as weighted relationships, making it suitable for graph databases and shortest-path algorithms. Neo4j [4] was used to store and query the transport graph, while Dijkstra's algorithm [5] was implemented to calculate the shortest available route between stations based on travel time. The system is a full-stack web application: graph theory and pathfinding on the backend, interactive visualisation on the frontend.

The development process followed an Agile Scrum methodology [6], organised into weekly sprints with planning and review meetings that let the team refine iteratively throughout the project. The system was divided into data ingestion, graph database design, routing logic, backend API development and frontend visualisation. The frontend uses React [7] and Next.js [8] for reusable components such as route panels, disruption controls and map overlays. Leaflet.js [9] and React-Leaflet provided interactive geographic visualisation for the map. On the backend, Neo4j represented the network graph while a dedicated routing layer handled route requests and disruption logic.

The TfL Unified API [10] is the primary data source. The station, line, route sequence and connection data were transformed into a graph compatible format for Neo4j, with preprocessing to normalise station identifiers and resolve duplicate entries.

The main goal was a working web system that shows the Underground network and finds alternative routes when a disruption is simulated.

2. Aims

The project set out to build, and then evaluate, an interactive disruption simulator for the London Underground. It was judged against five concrete objectives:

- Represent the network as a weighted graph whose edge costs reflect real station-to-station travel times.
- Compute time-optimal routes between any two stations while accounting for the real cost of changing lines, not just in-train time.
- Allow a user to impose station, line and single-connection closures and re-plan the affected journey interactively, fast enough to feel immediate.
- Run both hypothetical and live-ingested disruption through the same routing engine, so the two modes never diverge in behaviour.
- Remain usable when the live TfL feeds are incomplete or unreliable, degrading to estimated and simulated data rather than failing.

Success was assessed against these behaviours, measured re-planning latency, and feedback from a user study.

3. Background and Context

Several existing applications already support Underground journey planning, but they all assume one thing: the network as it is right now. TfL Go, the official app, gives live arrivals, status and step-free routing. Google Maps and Citymapper [11] mix Underground data with other modes to suggest the best current route. Every one of them answers "how do I get from A to B right now?" None of them let you ask "if the

Victoria line were suspended this evening, how much longer would my journey be, and where would I reroute?" That is the question our simulator was built to answer.

The formal representation of transport networks as graphs is well established in the research literature, particularly in studies of metro topology, efficiency, resilience and disruption response [12]–[15]. These approaches differ from the present work in both their unit of analysis and their mode of use. Latora and Marchiori [12] and Derrible and Kennedy [13] characterise networks through aggregate scalars, such as global efficiency or cycle-based robustness indicators. Rodríguez-Núñez and García-Palomares [14] use a GIS full-scan approach that weights links by riding time and counts missed trips, but removes one link at a time to rank elements for planners. Cats and Jenelius [15] model the value of real-time information during disruption, but their work remains an offline analytical model rather than a tool a user operates.

The novelty of this project does not lie in the underlying graph operation of removing nodes or edges and recomputing shortest paths, since that approach is already well established in research. Rather, it lies in applying this approach interactively to individual, user-defined London Underground journeys. The simulator recomputes a named origin-destination route as soon as users apply closure scenarios involving stations, lines or individual connections. The same routing engine is used for both hypothetical scenarios and live-ingested disruptions. The simulator therefore, bridges consumer journey planners and analytical resilience studies. It adopts the graph-based disruption modelling used in resilience research, but presents it through an interactive journey-planning interface.

The structure of the Underground makes it naturally suited to graph modelling: the simulator represents stations as vertices and adjacent station connections as weighted edges, allowing closures to be modelled by removing stations or connections from the graph. Once the network is expressed this way, the problem of finding the best journey becomes a shortest-path problem, and the question of how a closure affects journeys becomes the problem of removing vertices or edges and recomputing those paths. Dijkstra's algorithm provides the routing core of the system. It is appropriate for the London Underground because it calculates shortest paths in graphs with non-negative edge weights and performs well on networks of this size.

The A* algorithm [16] was considered as an alternative, since a geographic heuristic could also guide the search. With a binary-heap priority queue, Dijkstra's algorithm runs in $O((V + E) \log V)$. A* can reduce the number of nodes explored when it has an effective heuristic, but in this project any advantage was expected to be small because a straight-line geographic heuristic cannot account for the 300-second interchange

penalty and so prunes few nodes. For a network of a few hundred stations, Dijkstra meets the performance target without the added complexity, so it was retained.

We chose the database with the same graph thinking in mind. Relational databases can represent a network through join tables, but expressing repeated path queries over them is awkward. A graph database stores vertices and edges as first-class entities, which makes the model "stations connected by lines" explicit and keeps the persistence layer aligned with the conceptual model. Neo4j was selected for this reason: its property-graph model and Cypher query language [17] allow stations and their CONNECTS_TO relationships to be stored, constrained and retrieved directly, and the resulting structure maps cleanly onto the in-memory graph used for routing.

The data itself comes from the TfL Unified API, the authoritative and freely available source for line routes, station coordinates, live arrivals and line status. As later sections describe, the API is powerful but fragmented: no single endpoint supplies everything the simulator needs, and the live feeds vary in reliability. A lot of the work went into understanding what the data did and did not contain. We treated data ingestion as a modelling problem in its own right, not just calling the API and storing the response, and this became one of the project's main technical themes. The data pipeline, Neo4j persistence, API and React/Leaflet client layers are shown in Fig. 2.

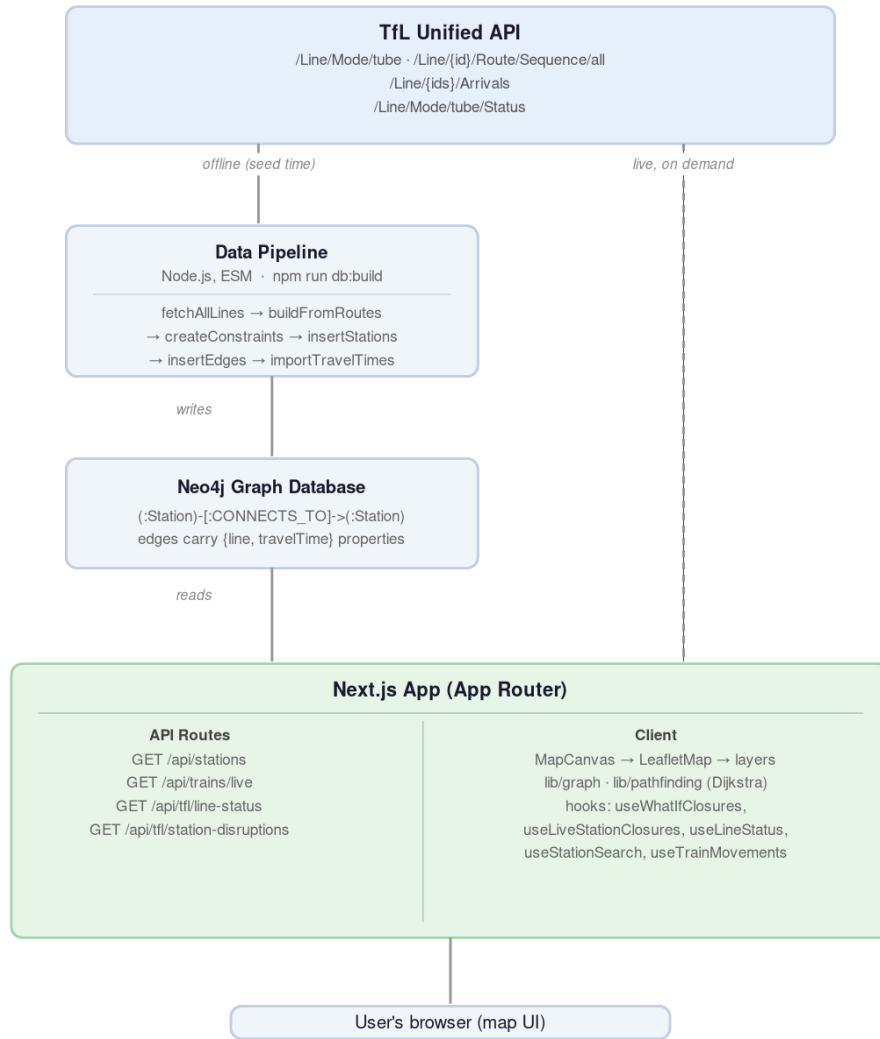


Fig. 2. High-level system architecture.

4. Key Technical Challenges

4.1 Frontend Visualisation and Performance

The interface was built on a design system, documented separately, which was established from a Figma moodboard before implementation and draws directly on Transport for London's visual heritage, Harry Beck's schematic tube map and the official line-colour palette, to keep the network immediately legible to Londoners. The application is dark-first and map-first: every control floats above a full-viewport map as a semi-transparent "glass" panel, so the network, live trains and the highlighted route remain visible while the user interacts with the controls. One early challenge was rendering multiple Underground lines sharing the same connection. As these lines used the same start and end coordinates, they were initially drawn directly on top of one another, causing only the uppermost line to be visible. This created a misleading visualisation, as users could not tell when more than one Underground line served the

same section of the network. To resolve this, small positional offsets were applied to overlapping line paths during rendering. This allowed shared connections to be visually separated while keeping the underlying graph data unchanged. As a result, the map became clearer and more accurate, particularly when users were inspecting routes or disruption scenarios involving shared sections of the network.

A significant frontend challenge involved the performance cost of displaying live train positions on the map. In the initial implementation, each train required several DOM elements to manage its icon, styling, and interaction state; visualising the full set consequently introduced approximately one thousand managed objects into the Leaflet layer, resulting in noticeable lag during panning, zooming and live data updates. The solution was a single HTML canvas element layered within the Leaflet map. Rather than maintaining an individual component tree for every train, the system draws all positions in a single canvas pass. This gives up Leaflet's built-in per-marker clicks, but keeps the object count almost flat no matter how many trains are shown. We added the click behaviour back with custom hit detection, described below. These positions are redrawn when the map moves, zooms, resizes or receives updated train data. Rendering is scheduled using `requestAnimationFrame`, reducing unnecessary redraws and aligning updates with the browser's rendering cycle.

Further optimisations were added by skipping trains outside the current viewport, hiding train markers below a defined zoom threshold, and allowing users to filter which lines display live trains. Custom hit detection was also implemented so users could still select trains despite them no longer being individual Leaflet markers.

The visual representation of disruption scenarios also required iteration. Closed stations, suspended lines and blocked connections needed to be clearly distinguishable without overwhelming the existing Underground line colours. Several approaches were considered, including opacity changes and colour-only indicators but these were less effective when multiple disruptions were active. The final design used red cross markers for closed stations and amber dashed overlays for suspended line segments, providing visual distinction while preserving map readability as shown in Fig. 3.



Fig. 3. Disruption indicators: red crosses for closed stations and amber dashed overlays for suspended segments.

The system also needed to display calculated routes on top of the base network. Early route overlays were difficult to distinguish from existing line colours, particularly in dense areas where several services run close together. The final implementation uses a bright green route highlight with a dashed centre line, rendered above the standard network layers. Route stations are emphasised using larger green circular markers. Fig. 4 shows a calculated route with its information panel.

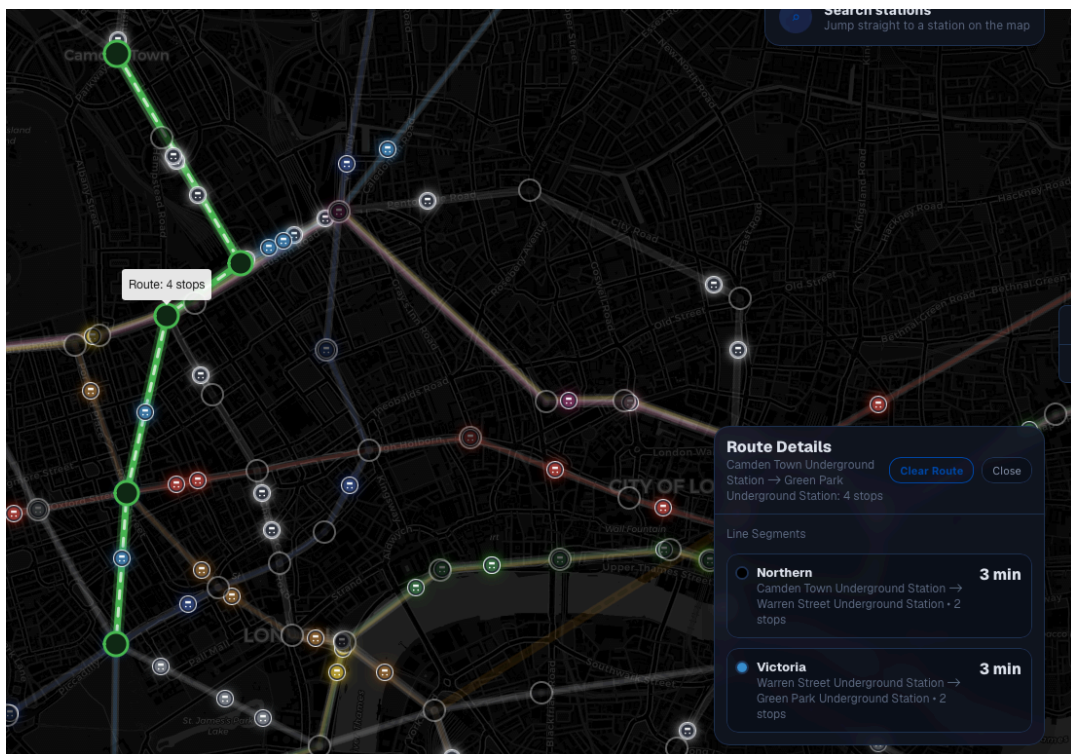


Fig. 4. A computed journey highlighted in green with the route information panel.

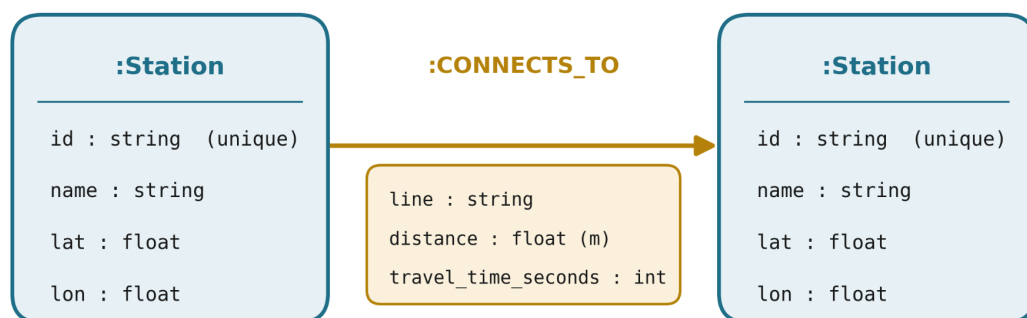
This makes the selected journey visually distinct while allowing the underlying map and surrounding transport network to remain visible.

4.2 Data Ingestion and Graph Modelling

A challenge during data ingestion was that the TfL route API did not provide all of the data needed for realistic routing and train simulation. The route sequence endpoints were useful for identifying the order of stations on each Underground line, which allowed the system to create Station nodes and CONNECTS_TO relationships in Neo4j. However, these endpoints did not include station-to-station travel times. To solve this, additional TfL timetable data was used. TfL provides timetable information through XML files, which were stored in the repository under “travel_time_data”. These files contained journey pattern timing links, including the runtime between adjacent platform stop points. A separate Python extraction script was written to process these XML files and convert the timing information into a simpler CSV format.

The extraction process involved mapping platform-level stop identifiers to station-level identifiers. TfL timetable files often refer to individual platforms, whereas the graph model that we crafted represents stations as single nodes. The script converted platform IDs into the same station ID format used by the Neo4j graph, then extracted the runtime between consecutive stop points. Where multiple timings existed for the same station-to-station connection, the shortest valid runtime was kept. This produced the CSV file: “travel_time_data/tfl_station_travel_times.csv”.

During the database seed process, the main graph is first built from TfL route data, creating the station nodes and CONNECTS_TO relationships. The travel-time CSV is then imported afterwards and matched against those relationships, adding a travel_time_seconds property. Fig. 5 shows the resulting graph data model.



Relationships are created bidirectionally; travel_time_seconds is the edge weight used by routing.

Fig. 5. The Neo4j data model: Station nodes joined by weighted CONNECTS_TO relationships.

This allowed the graph to combine route structure from the TfL API with realistic travel-time data from the XML timetable files. To prevent graph gaps and broken routes when exact travel times were unavailable, a distance-based fallback weight was used. This ensures full network connectivity, prioritising routable journeys over perfectly precise timing on the minority of unmatched edges. Overall, this made the graph model more useful than a simple station map. The CONNECTS_TO relationships could be weighted by travel time, supporting more realistic shortest-path routing, route summaries and train movement simulation.

4.3 Routing, State Management and Backend Reliability

Routing is the technical core of the simulator and the routing algorithm runs on the client rather than the server. The full station graph is fetched once from the backend, cached and converted into an in-memory adjacency list by `buildGraph`, after which Dijkstra's algorithm executes entirely in the browser.

A server-side architecture would require a network round-trip for every computation, adding latency. The What-If simulator is built for rapid experimentation, where users toggle multiple disruptions in sequence so that per-request overhead would make the system feel sluggish. A client-side architecture was selected to avoid the latency inherent in server-side requests: by caching the entire network graph once on launch, routing is restricted only by the performance of the local Dijkstra implementation (averaging 1.8 ms, Section 7.1), ensuring that re-planning remains instantaneous as disruption scenarios are toggled. This kept re-planning inside the ~100 ms threshold at which interaction feels instantaneous, which a per-request server round-trip could not guarantee.

A plain shortest-path search over stations was not sufficient for realistic journeys, because it cannot represent which line a passenger is currently travelling on, and therefore cannot account for the real cost of changing lines. To solve this, the algorithm expands each station into a set of per-line states, keyed as "{stationId}_{line}", with a dedicated START state for the origin. The search then tracks not only where a passenger is but which line they arrived on. When a relaxed edge belongs to a different line from the current state, the algorithm adds a fixed change penalty of `CHANGE_PENALTY_SECONDS`, set to 300 seconds (five minutes), on top of the edge's travel-time weight. This value is consistent with the lower bound of the 5–15 minute interchange penalty range identified in the transport literature and informed by

supervisor guidance [18]. This discourages journeys that minimise in-train time but demand many awkward interchanges, producing routes that better reflect how passengers actually travel. The final path is reconstructed from the cheapest end-state back to the origin, and the engine returns the ordered list of stops, the total journey time including penalties, and the number of line changes.

The same algorithm provides the What-If capability through three exclusion sets passed into the search: closed stations, closed lines and blocked individual connections. Closed stations are skipped during edge relaxation unless they are the journey's destination; closed lines remove every edge tagged with that line; and blocked connections are matched against an undirected from-to-line key so that a single severed link can be modelled without removing an entire line. These sets are supplied either by the user's hypothetical selections or, in Live mode, by genuine closures and line-status data, so the same routing logic serves both simulated and real disruption. This mode distinction is reinforced visually. Entering What-If Mode shifts every interactive accent from blue to amber and frames the viewport with corner brackets and a "WHAT-IF" label, so a hypothetical closure can never be mistaken for ordinary route planning. The visual identities of the modes are compared in Fig. 6.

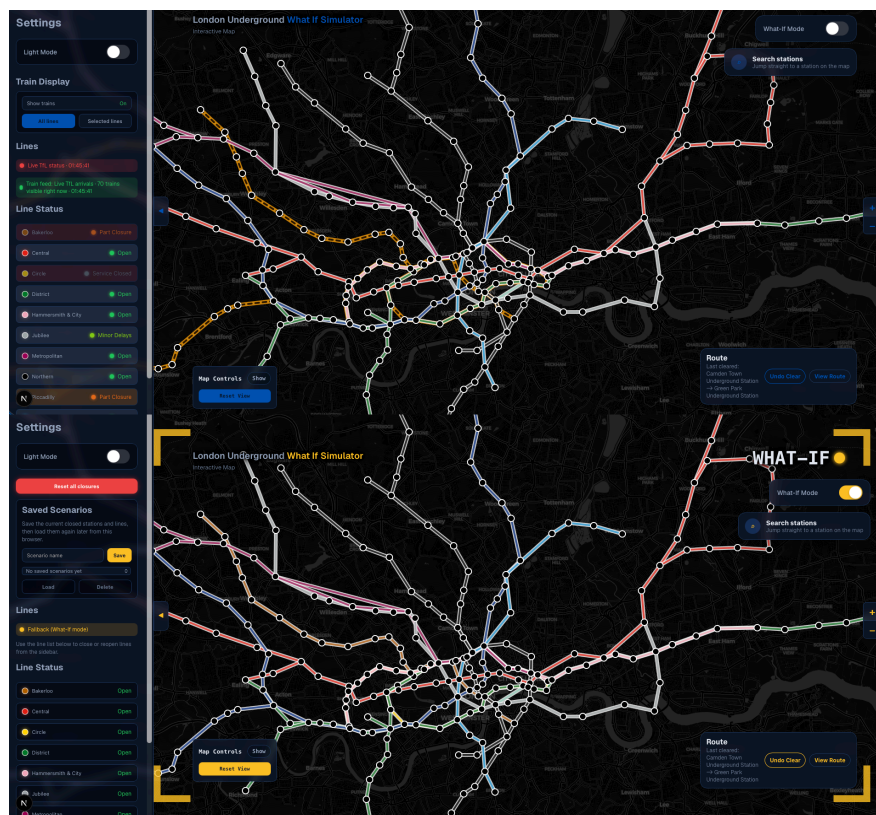


Fig. 6. The accent-colour mode signal: Live mode (blue) and What-If mode (amber), with corner brackets and overlay label marking the simulated state.

When no path exists under the active constraints, the engine reports failure cleanly and the interface surfaces a routing-error message rather than presenting an empty or misleading result.

State management uses focused custom hooks instead of a global Redux store. Since state divides along feature lines with no cross-cutting slices, dedicated hooks avoid unnecessary boilerplate while keeping concerns predictable and testable.

The *useWhatIfClosures* hook manages closed stations and lines, guarding mutations with a mode check to prevent accidental toggling in Live mode. It supports persistent storage for named disruption scenarios, using read-time normalisation to prevent data corruption. The *useRoutePanel* hook handles routing results and panel visibility. This modular approach simplifies reasoning about the interplay between routing logic and live overlays.

Backend reliability was addressed through caching, request de-duplication and clear error semantics, since the application depends on both its own Neo4j database and the external TfL API. The GET /api/stations endpoint assembles the network graph from Neo4j, de-duplicates stations into a node map, converts Neo4j integer values into safe JavaScript numbers, and caches the assembled result for one hour. It also collapses concurrent requests into a single in-flight query, so that several simultaneous map loads trigger only one database read rather than many. Endpoints serving as proxies for the TfL API, line status and station disruptions are cached, whereas live arrival data is retrieved uncached to ensure maximum freshness; in instances of upstream failure, all three services emit clear HTTP 502 errors. This lets the client distinguish an external outage from an internal error and respond appropriately.

The biggest reliability problem was the live train feed, which was never relied on as a guaranteed real-time source. The feed service requests arrivals from the TfL API and filters out untrackable predictions, such as records carrying the placeholder vehicle identifier 000. It then constructs a stable per-train key from the line and vehicle identifiers. As the live train feed frequently provides inconsistent vehicle identifiers across polling intervals and records often vanish between sequential API requests, the system cannot guarantee continuous, uninterrupted tracking of individual trains. The *useTrainMovements* hook therefore merges live snapshots when reliable live data is available and falls back to simulated train movement, derived from the stored graph and its travel-time weights, when it is not. A feed-status object records whether the current display is live or simulated, so even when the TfL data is patchy or drops out, the map stays populated and the tool keeps working.

5. User Research

A formal user survey was conducted to evaluate the prototype during the final stages of the project. This section covers participant demographics, the data collected and the design refinements made in response. While a comprehensive analysis involving qualitative coding and data visualisations is maintained in a separate user study report, the following provides a condensed overview for this technical report.

5.1 Survey Data

The survey used a convenience sample of 21 respondents, including students and commuters with diverse familiarity with the London Underground. The demographic skewed younger, with fifteen participants aged 18–24. Self-reported network familiarity was varied: 7 were "Not Familiar", 6 "Slightly Familiar", 5 "Moderately Familiar", and 3 "Very Familiar". Technical proficiency was generally high, with 12 "Intermediate" and 4 "Advanced" users and the remaining five reporting a "Beginner" or "Non-technical" background. All participants were aged 18 or over and provided consent for the anonymous use of their data for academic research.

The questionnaire used fifteen 5-point Likert statements [19], covering how well tasks could be completed, how useful the features were and how clear and usable the interface felt. This was supplemented by three open-ended questions targeting system strengths, areas for improvement and potential bugs.

Key findings: Sentiment was largely positive with most Likert items receiving "Agree" or "Strongly Agree" ratings as shown in Fig. 7.

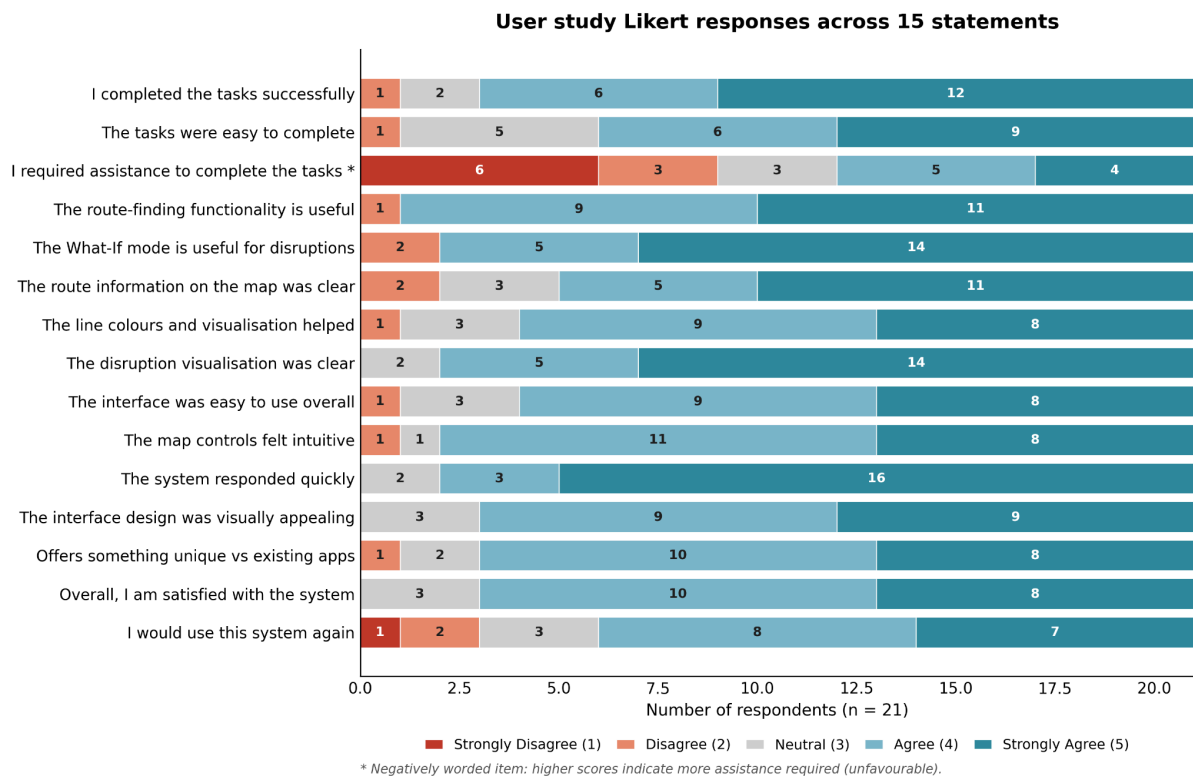


Fig. 7. Distribution of Likert responses across the fifteen statements (n = 21).

Theme means ranged from 3.86 to 4.45, all above the scale midpoint. The proportion rating 4 or 5 was strongest for the two routing features: route-finding usefulness reached 95.2% and the What-If mode 90.5%, suggesting the feature was well received once used.

The strongest insight was that the What-If mode was both the simulator's main value proposition and its clearest usability weakness. It was the most frequently mentioned positive feature in the open-ended responses, cited by eight respondents, and achieved 90.5% top-two-box agreement for usefulness. However, seven respondents also reported difficulty finding it, indicating that the feature was valued once discovered but hidden too deeply in the interface. This interpretation was reinforced by qualitative and semantic analysis of the open-ended responses, where phrases around "what if", "find" and "easier" repeatedly appeared in improvement suggestions. This discoverability issue informed significant UI refinements. Additional feedback highlighted the need for a light theme to improve contrast; minor UI overlaps between panels and requests for extended network coverage including the Elizabeth Line and buses. The full breakdown is in Table I in the Appendix.

These results should be read with the study's scope in mind. The sample was a convenience sample of 21 participants skewed toward 18–24-year-olds with above-average technical proficiency, so it is not representative of the wider commuter

population and the findings are indicative rather than statistically generalisable. Reported figures are descriptive only; no inferential testing was performed and the small sample makes the per-item proportions sensitive to a few responses. Self-reported Likert ratings may also carry mild positivity bias given the supervised demonstration setting. A sample skewed toward technically proficient younger users also likely understates the difficulty that older or non-technical commuters would face with the dense, dark-first map interface, leaving its usability for that group effectively untested, a gap the planned high-contrast mode and WCAG 2.1 AA work (Section 7.3) only partly addresses.

The participants were students and commuters rather than the transport planners, analysts and researchers named as potential beneficiaries, so the usability evidence here speaks to general users; the system's value for professional analytical use remains a design expectation grounded in the feature set rather than a tested result.

5.2 Design changes prompted by user feedback

The study findings prompted five key iterative changes during the project's final development phase. A sixth recommendation, involving support for the Elizabeth, buses and platform-level detail, was moved to the future roadmap in Section 7.3 due to project constraints.

The first change involved improving the discoverability of the What-If mode. This was the primary usability issue identified with the feature, with 7 of 21 respondents reporting that the panel was difficult to locate. The original placement in the side drawer was replaced with a persistent main map control in the right corner, shown in Fig. 8.

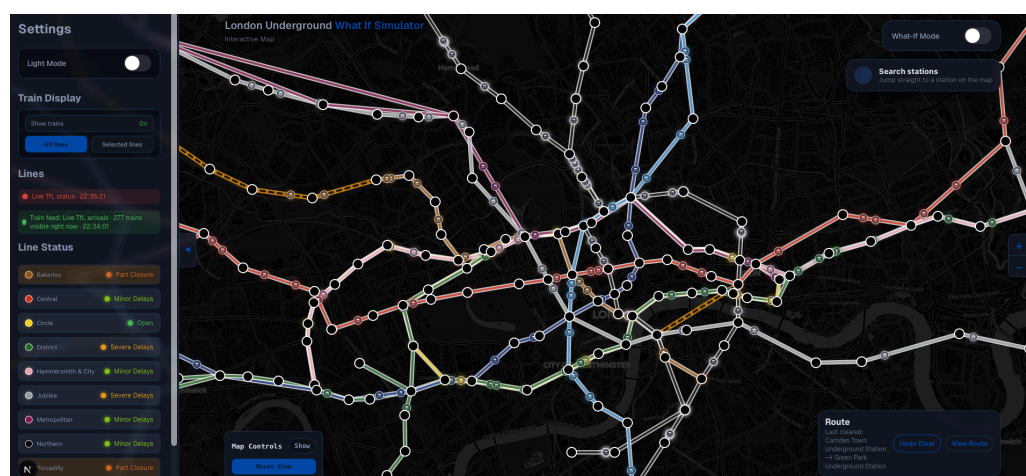


Fig. 8. The relocated What-If control and disruption panel.

An in-map prompt was added after the user completes their first route search, encouraging them to try a disruption scenario.

The second change was the implementation of a light theme. Five users specifically requested a lighter visual mode to address contrast and readability issues. Fig. 9 compares the two themes.

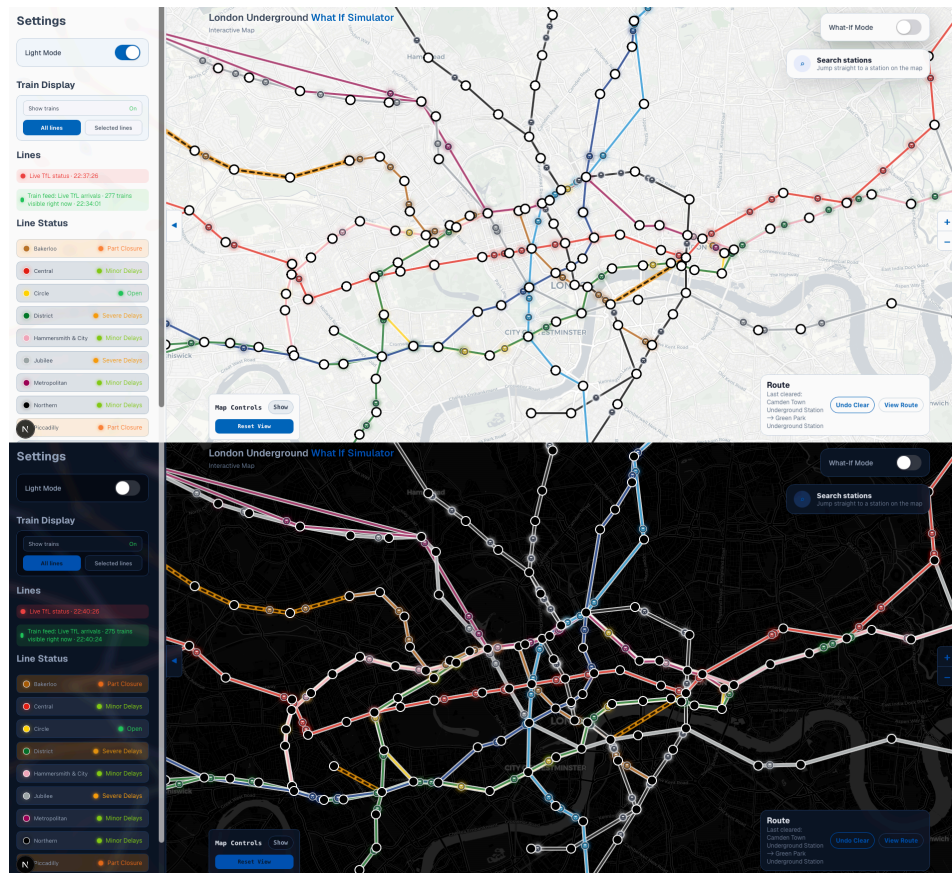


Fig. 9. The interface in dark and light themes.

Alongside this addition, the team reviewed dark-mode contrast levels to improve label legibility and address concerns that station markers could occasionally obscure text.

The third change resolved a set of panel-interaction and selection bugs. Several respondents identified interaction issues, including overlapping search boxes, accidental station clearing after double-clicks and routes being cleared after unintended map clicks. Each was fixed during the final development phase: the search boxes were repositioned to remove the overlap, double-click handling was corrected so selections are no longer cleared unintentionally, and map-click handling was changed to preserve the active route. These fixes were committed and verified against the routing-state and panel test suites, improving the predictability and reliability of the interface.

The fourth change was the addition of an integrated onboarding sequence. To reduce initial interface complexity, a guided tour was added on first launch. This onboarding sequence introduces the main route-finding workflow and explicitly explains the What-If functionality, directly addressing the discoverability issue identified in the survey.

The fifth change was automatic route framing. Following a user suggestion that the map should adjust after a route is generated using the search panel, an auto-fit behaviour was implemented. The view now dynamically centres and zooms to display the full journey route once both the start and destination stations are selected.

6. Project Outcomes and Reflection

6.1 Team Dynamics

The project was developed using a collaborative Agile workflow, with responsibilities distributed across the team according to both individual strengths and sprint priorities. Rather than assigning fixed roles for the entire project, tasks evolved dynamically depending on the features being developed at each stage. That kept the team flexible as whoever was free picked up whatever the current sprint needed, whether that was frontend, routing or the data pipeline.

Team communication was maintained through regular meetings and sprint planning. These sessions allowed the group to agree priorities, identify blockers and coordinate work across different parts of the system. Version control was managed through feature branches, enabling individual team members to work independently while still integrating their contributions into the shared codebase through review and testing.

The team also encountered several technical disagreements during development, particularly around interface behaviour and visual clarity within the map system. One recurring discussion involved the rendering order of train icons and station nodes. Some team members preferred train markers to remain visually prioritised to improve live train visibility, while others argued that this obscured station interactions. After discussion and testing, the rendering layers were adjusted to balance train visibility with station accessibility.

A further design issue emerged during supervisor feedback, when it was noted that the live train markers appeared too visually similar to station icons. This made it harder for the users to quickly distinguish between fixed stations and moving trains during map interaction. In response, the train icons were redesigned with clearer visual styling, improving readability and reducing potential user confusion.

The team worked well together, mainly because disagreements got raised openly and settled by trying things out rather than arguing in the abstract.

6.2. Key Insights

One key insight from the project was that the TfL data ecosystem is both powerful and fragmented. The TfL API provided useful route, station, disruption and live status data, which made it possible to build features beyond a static route planner. However, not all required data was available from a single endpoint. For example, station ordering could be retrieved from the route API but station-to-station travel times had to be extracted separately from TfL timetable XML files.

Another important finding was that the raw TfL data model was more detailed than the application required. TfL often represents stations through a hierarchy of hubs, platforms, entrances and stop points. While this is useful for transport data accuracy, it introduced unnecessary complexity for route calculation. The project therefore benefited from simplifying the data into a graph model where each station became a node and direct station-to-station links became weighted relationships. This abstraction made routing and simulation easier while still preserving the important structure of the Underground network.

The project also showed that live transport data should be treated as best-effort rather than perfectly reliable: as described in Section 4.3, inconsistent train identifiers and dropped records made continuous tracking unreliable, so the system combined live data with estimated timings and fallback simulation to stay usable.

Overall, the project demonstrated that the main challenge was not only building the interface or routing algorithm but designing a reliable pipeline around real-world transport data. The final system had to pull together several data sources and flatten TfL's complex structures into something routable. It also needed sensible fallbacks for when live data was missing.

6.3 Iterative Changes

Iteration was a constant feature of the project rather than a single phase, and two distinct rounds stand out. The rendering round of Section 4.1 (the canvas train layer, the disruption indicators and the route highlight) was driven internally, by profiling and developer inspection. The usability round of Section 5.2 (What-If discoverability, the light theme, panel-interaction fixes, onboarding and automatic route framing) was driven externally, by the late user study. The difference taught us that the internal problems showed up as soon as we built a feature, but the usability ones stayed hidden until real users tried the interface.

The development process itself was built to support this kind of iteration. Work was organised into weekly Agile sprints, with planning and review meetings that let the team adjust scope as problems emerged, and version control was managed through feature branches that allowed routing, disruption, visualisation and data-pipeline work to proceed in parallel before being integrated through review. An automated test suite covering the graph construction, the pathfinding logic and the interface components gave the team the confidence to refactor and rework features without silently breaking existing behaviour. The main reflection on this process is that usability testing came relatively late: several interface issues only became visible during the user study and conducting that evaluation earlier would have allowed the same iterative improvements to be made at lower cost.

7. Conclusion

This project delivered an interactive London Underground "What-If" simulator. It brings live transport data and graph-based routing together behind a map-based, full-stack web application. The final system allows users to explore the Underground network, calculate routes between stations and simulate disruption scenarios such as closed stations and blocked connections. This met the project's main objective: going beyond standard journey planners by letting users see how changes to the network affect which routes are available and how long they take.

Each of the five aims set out in Section 2 was met. Table II in the appendix maps each objective to its outcome and the evidence supporting it.

7.1 Successes

Several aspects of the project were particularly successful, beyond the core integration of TfL data, Neo4j and Dijkstra-based routing described above.

The performance was also a success. Route calculations completed within the ~100 ms threshold below which an interface feels instantaneous to the user [20], across the full Underground network, making re-planning feel immediate during interactive use. To verify these claims, route computation was timed with `performance.now()` around the client-side dijkstra call over a sample of 50 random origin–destination pairs spanning the full network, run on a 2025 MacBook Pro M4 Pro in Chrome Version 148. Live-train rendering was sampled by counting `requestAnimationFrame` callbacks over one second with the full ~500-train set visible. Station-graph API latency was read from the browser Network panel for the first (uncached) and a repeat (cached) request.

The frontend was also optimised through canvas-based train rendering to reduce the cost of displaying large numbers of live trains. With a mean route computation time of

1.8 ms and a worst-case latency of 5.7 ms, every measured route fell well inside the ~ 100 ms perceptual-immediacy threshold: the mean is roughly fifty times faster than that threshold, and even the worst case sits an order of magnitude inside it. This confirms that the decision to run Dijkstra on the client is what keeps interactive What-If re-planning feeling instantaneous. The canvas-based train layer maintained a steady ~ 60 fps while rendering the full train set, while the one-hour station-graph cache effectively reduced repeat load times from 26 ms to 1 ms. The canvas layer replaced an earlier Leaflet-marker layer that had fallen below acceptable frame rates when managing ~ 500 trains. As these figures came from one high-specification machine, absolute timings would be slower on weaker hardware, but the large margin leaves the interactive-routing claim intact. The full results are detailed in Table III: Measured performance figures in the Appendix.

The project also benefited from a structured development process. Weekly sprints helped the team prioritise work and adjust scope when necessary. Testing was incorporated throughout the project through unit, user and integration tests, supported by our automated test suites. In total, the system is verified by 95 automated tests (32 backend and 63 frontend, across 23 suites) alongside 30 manual test cases covering map rendering, routing, disruption flows, the live train feed and mobile responsiveness. All tests pass, with a single opt-in Neo4j integration test skipped by default as it requires a live database. A suite-level breakdown is given in Table IV in the Appendix, with full detail in the project's Full Testing Report. This helped identify regressions early and improved confidence in the reliability of the routing, disruption and interface features.

The simulator bridges the gap between the live journey planners people use day to day, and the resilience models used in research. It combines the interactivity of consumer route-planning tools with the remove-and-recompute approach used in transport-network research, allowing users to test closures and immediately see their effect on a specific Underground journey. Its current limits, including Underground-only coverage and fixed interchange costs, reflect the project scope and provide clear directions for future work.

7.2 Learnings

The project provided valuable experience in applying computer science concepts to a real-world transport domain: modelling the Underground as a graph showed how theoretical ideas such as nodes, weighted edges and shortest-path algorithms solve a practical routing problem. Equally, it gave the team hands-on experience with technologies new to several members, particularly Neo4j, graph database design and full-stack integration, and reinforced the value of designing for iteration, since the

strongest interface improvements emerged only from user feedback rather than upfront planning.

7.3 Future Work

If the project were continued over the next year, several extensions would be valuable. The routing engine could be expanded to include A* pathfinding using a geographic heuristic, which may improve efficiency as the network model grows. As discussed from our user testing survey, multi-modal routing could be introduced by incorporating the Overground, Elizabeth line and bus connections, allowing users to not only simulate disruption over the underground but London as a whole.

Further work could also explore passenger flow simulation using crowding or demand data, enabling the system to recommend routes that are not only shortest but also less congested. Historical disruption data could be used to support predictive modelling, helping estimate which closures or delays are more likely at particular times of day or week.

Finally, the user experience could be improved through a dedicated mobile application with offline map support, as commuters may need access while travelling through areas with poor connectivity. Additional accessibility features, including improved screen reader support and a high-contrast mode aligned with WCAG 2.1 AA standards [21], would also make the system more inclusive and suitable for wider deployment.

8. Acknowledgements

The authors would like to thank Peter Rodgers for his guidance and support throughout the development of this project. We also thank the participants who completed our user survey and provided feedback during testing and demonstration sessions. Their responses helped inform several improvements to the system.

9. References

- [1] Transport for London, "Underground services performance." [Online]. Available: <https://tfl.gov.uk/corporate/publications-and-reports/underground-services-performance>. [Accessed: 30 May 2026].
- [2] Transport for London, "TfL Go app — Maps, routes and more to plan your journey." [Online]. Available: https://tfl.gov.uk/maps_/tfl-go. [Accessed: 30 May 2026].
- [3] Google LLC, "Google Maps." [Online]. Available: <https://www.google.com/maps>. [Accessed: 30 May 2026].
- [4] Neo4j, Inc., "Neo4j Graph Database — Documentation." [Online]. Available: <https://neo4j.com/docs/>. [Accessed: 30 May 2026].
- [5] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [6] K. Schwaber and J. Sutherland, "The Scrum Guide," 2020. [Online]. Available: <https://scrumguides.org/>. [Accessed: 30 May 2026].
- [7] Meta Open Source, "React Documentation." [Online]. Available: <https://react.dev/>. [Accessed: 30 May 2026].
- [8] Vercel, Inc., "Next.js Documentation." [Online]. Available: <https://nextjs.org/docs>. [Accessed: 30 May 2026].
- [9] V. Agafonkin, "Leaflet — an open-source JavaScript library for interactive maps." [Online]. Available: <https://leafletjs.com/>. [Accessed: 30 May 2026].
- [10] Transport for London, "TfL Unified API," TfL API Portal. [Online]. Available: <https://api-portal.tfl.gov.uk/>. [Accessed: 30 May 2026].
- [11] Citymapper Limited, "Citymapper." [Online]. Available: <https://citymapper.com/>. [Accessed: 30 May 2026].
- [12] V. Latora and M. Marchiori, "Is the Boston subway a small-world network?," *Physica A*, vol. 314, no. 1–4, pp. 109–113, 2002.
- [13] S. Derrible and C. Kennedy, "The complexity and robustness of metro networks," *Physica A*, vol. 389, no. 17, pp. 3678–3691, 2010.

- [14] B. Rodríguez-Núñez and J. C. García-Palomares, "Measuring the vulnerability of public transport networks," *Journal of Transport Geography*, vol. 35, pp. 50–63, 2014.
- [15] O. Cats and E. Jenelius, "Dynamic vulnerability analysis of public transport networks: mitigation effects of real-time information," *Networks and Spatial Economics*, vol. 14, no. 3–4, pp. 435–463, 2014.
- [16] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [17] Neo4j, Inc., "Cypher Query Language." [Online]. Available: <https://neo4j.com/docs/cypher-manual/current/>. [Accessed: 30 May 2026].
- [18] M. Wardman and J. Hine, "Costs of Interchange: A Review of the Literature," Working Paper 546, Institute of Transport Studies, University of Leeds, Leeds, UK, Jul. 2000. [Online]. Available: <https://eprints.whiterose.ac.uk/id/eprint/2075/>. [Accessed: 30 May 2026].
- [19] R. Likert, "A technique for the measurement of attitudes," *Archives of Psychology*, vol. 22, no. 140, pp. 1–55, 1932.
- [20] J. Nielsen, "Response Times: The 3 Important Limits," in *Usability Engineering*. San Francisco, CA, USA: Morgan Kaufmann, 1993. [Online]. Available: <https://www.nngroup.com/articles/response-times-3-important-limits/>. [Accessed: 30 May 2026].
- [21] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Online]. Available: <https://www.w3.org/TR/WCAG21/>. [Accessed: 30 May 2026].

Appendix A. User Testing Questionnaire Results Table

Table I: User study questionnaire results table (n = 21).

Statement	Strongly Disagree (1)	Disagree (2)	Neutral (3)	Agree (4)	Strongly Agree (5)

I was able to complete the tasks successfully.	0	1	2	6	12
The tasks were easy to complete.	0	1	5	6	9
I required assistance to complete the tasks. *	6	3	3	5	4
The route-finding functionality is useful.	0	1	0	9	11
The What-If mode is useful for simulating disruptions.	0	2	0	5	14
The route information displayed on the map was clear.	0	2	3	5	11
The Underground line colours and visualisation were helpful.	0	1	3	9	8
The disruption visualisation was clear and understandable.	0	0	2	5	14
The interface was easy to use overall.	0	1	3	9	8
The map controls and navigation felt intuitive.	0	1	1	11	8

The system responded quickly during use.	0	0	2	3	16
The interface design was visually appealing.	0	0	3	9	9
Compared to existing transport applications, this system offers something unique.	0	1	2	10	8
Overall, I am satisfied with the system.	0	0	3	10	8
I would use this system again.	1	2	3	8	7

* This item uses negative wording – higher scores indicate more required assistance, representing an unfavourable response. All other items follow the convention where higher scores indicate greater favourability.

Table II: Aims against outcomes.

Aim (Section 2)	Outcome	Evidence
1. Weighted graph with edge costs from real station-to-station travel times, not hop counts	Met	Section 4.2; Fig. 5; graph ingestion test suites (Table IV)
2. Time-optimal routes that account for the real cost of changing lines, not just in-train time	Met	Section 4.3 (per-line states, 300 s change penalty); pathfinding suite, 13 tests (Table IV)
3. User-imposed station, line and single-connection closures,	Met	Section 4.3 (three exclusion sets); Section 7.1 and Table III (1.8 ms mean, inside the 100 ms threshold)

re-planned interactively and immediately		
4. One routing engine driving both hypothetical and live-ingested disruption, with no divergence	Met	Section 4.3 (shared exclusion logic for simulated and live modes; Fig. 6)
5. Graceful degradation when live Tfl feeds are incomplete or unreliable, rather than failing	Met	Section 4.3 (live/simulated fallback, feed-status object, HTTP 502 semantics); Section 4.2 (distance-based fallback weight)

The two further success criteria named in Section 2 are reported separately: re-planning latency in Section 7.1 and Table III, and user-study feedback in Section 5.

Table III: Measured performance figures.

Metric	Method	Result
Single route, mean of 50 pairs	timed dijkstra	1.8 ms
Worst-case route (slowest of 50)	timed dijkstra	5.7 ms
Live train layer, ~500 trains (canvas)	rAF frames/sec	63 fps
GET /api/stations, cold vs cached	Network panel	26 ms / 1 ms

Table IV: Automated test suite breakdown.

Area	Layer	Suites	Tests
Graph construction and validation	Backend	2	11
Data ingestion and travel-time import	Backend	1	5
Pathfinding (Dijkstra, change penalty, exclusions)	Backend	1	13
Stations API route	Backend	1	3
Routing state, panels and search UI	Frontend	5	15
Live train feed and movement	Frontend	8	31
Line status	Frontend	1	3
Map rendering (markers and layers)	Frontend	2	5
Shared utilities	Frontend	2	9
Total (run by default)		23	95
Neo4j integration (opt-in, skipped by default)	Backend	1	1

*The Neo4j integration suite is skipped unless RUN_NEO4J_TESTS=true, as it requires a live database; the 23 suites and 95 tests above are those run by default, all passing.